

2 MOTIVATING EXAMPLE

Here is a small example where we construct a prompt (in blue) which consists of a brief instruction, and the diarization transcript with several errors. The prompt is sent to Google Bard [36] via its Web UI¹. We can see that the completion generated by LLM (in orange) correctly moved the error words to the right speakers. This example can be easily reproduced with many off-the-shelf commercial or open source instruction-finetuned LLMs, such as Google Bard [36] or OpenAI ChatGPT [37].

Prompt:

In the speaker diarization transcript below, some words are potentially misplaced. Please correct those words and move them to the right speaker. Directly show the corrected transcript without explaining what changes were made or why you made those changes.

<spk:1> Good morning Patrick, how
 <spk:2> are you? Good, good. How are you Tom? Pretty
 <spk:1> good. Going to work?
 <spk:2> Yes. Busy day. How are your kids? Do they go
 <spk:1> to school? Oh they are too young for that. I sent them to daycare earlier
 <spk:2> today. Oh yeah I forgot about that.

Completion:

<spk:1> Good morning Patrick, how are you?
 <spk:2> Good, good. How are you Tom?
 <spk:1> Pretty good. Going to work?
 <spk:2> Yes. Busy day. How are your kids? Do they go to school?
 <spk:1> Oh, they are too young for that. I sent them to daycare earlier today.
 <spk:2> Oh yeah, I forgot about that.

Apart from correcting diarization errors, we also found several other diarization capabilities of LLMs, such as autofilling speaker names, autofilling speaker roles, or even completely replacing the orchestration module. Examples of these capabilities are shown in Appendix B. However, due to lack of relevant training and testing datasets, these capabilities are not thoroughly studied or evaluated in this paper. Here we focus on LLM’s capability of reducing diarization errors.

3 DIARIZATIONLM

3.1 SYSTEM OVERVIEW

We illustrate the DiarizationLM framework in Fig. 2. In this framework, the ASR and speaker diarization systems are frozen, and their outputs are processed by the orchestration module to associate a speaker label with each recognized word. The orchestrated diarization outputs are processed by a *prompt builder* module, which creates a compact textual representation of the diarized transcript, segment it into shorter versions to fit the LLM input size limit, and apply prompt prefix and suffix. The prompts are then sent to a finetuned LLM, and the completions generated by the LLM will be handled by a *completion parser* module, which truncates undesired outputs from the LLM, combines the completions of multiple segments, and apply a transform (see Section 3.4) to preserve the original transcripts of the ASR model.

3.2 PROMPT BUILDER

The output of the orchestration module is two sequences of equal length: a sequence of words, and a sequence of speaker labels. To fit it into a prompt, we use a compact textual representation, where speaker tokens are only inserted in the beginning of the transcript, or when the speaker has changed. Below is an example:

¹We used an internal version of Bard that is based on a larger model and supports more tokens than the public version.

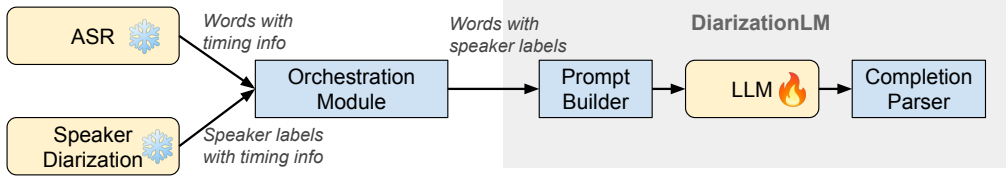


Figure 2: Diagram of the proposed DiarizationLM framework.

Example text representation of the prompt

Word sequence: ["good", "morning", "how", "are", "you"]
 Speaker sequence: [1, 1, 2, 2, 2]
 Text representation: "<spk:1> good morning <spk:2> how are you"

Since most LLMs have an input length limit, the text representation of an entire utterance may not fit this limit. In such cases, we recursively binary partition the word and speaker sequences in the middle, until all segments fit the the input length limit.

We also apply prefix and suffix to each prompt. The prefix is usually an instruction describing the task for the LLM to perform, and the suffix is a sequence of tokens to indicate the end of the prompt.

3.3 COMPLETION PARSER

Each prompt from the prompt builder will be sent to the finetuned LLM, which will generate a text completion for this prompt. First of all, we need to truncate any undesired outputs from the LLM. For example, during the LLM finetuning, each completion may have a suffix to indicate the end of the completion. Thus the suffix and any text generated after the suffix should be truncated from the original completion.

After the truncation, we need to convert the text representation of the completion back to the word sequence and the speaker sequence format. If the text representation does not start with a speaker token, we either use the last speaker from the previous segment, or just use speaker 1 if it is the first segment.

Next, we concatenate the word sequences and speaker sequences from all segments. However, the resulting concatenated word sequence may not be identical to the original word sequence from the ASR model due to modifications by LLM. This is undesired and may hurt word error rate. Thus here we need an algorithm to transfer the speaker labels from the concatenated speaker sequence to the original word sequence from the ASR model. We will introduce this algorithm in the following section.

3.4 TRANSCRIPT-PRESERVING SPEAKER TRANSFER

Here we describe an algorithm called *Transcript-Preserving Speaker Transfer* (TPST), which will be used in several places in our proposed framework, including training data preparation and the completion parser module.

Assume we have two sets of diarized transcript, referred to as “source” and “target”, each represented by two sequences of the same length: a sequence of words, and a sequence of speaker labels. The purpose of TPST is to transfer the speaker labels from the source sequences to the target sequences, such that:

1. The transferred speaker label sequence has a 1-to-1 association with the target word sequence.
2. The transferred speaker labels are more consistent with the source speaker labels.

As an example, the concatenated word sequence from the completion parser module may not be identical to the original word sequence from the ASR model. Thus we can treat the completion sequences as the source, and the original sequences from the orchestration module as the target, and transfer the speaker labels. Finally, the DiarizationLM outputs will be the original word sequence, associated with the transferred speaker label sequence.

The detailed TPST algorithm is described in Algorithm 1. An implementation is open sourced on GitHub².

Algorithm 1 The transcript-preserving speaker transfer (TPST) algorithm.

inputs
 Source word sequence of length N : $\mathbf{w}^{src} = (w_1^{src}, \dots, w_N^{src})$
 Source speaker sequence of length N : $\mathbf{s}^{src} = (s_1^{src}, \dots, s_N^{src})$
 Target word sequence of length M : $\mathbf{w}^{tgt} = (w_1^{tgt}, \dots, w_M^{tgt})$
 Target speaker sequence of length M : $\mathbf{s}^{tgt} = (s_1^{tgt}, \dots, s_M^{tgt})$

outputs
 Transferred speaker sequence of length M : $\mathbf{s}^{tra} = (s_1^{tra}, \dots, s_M^{tra})$

- 1: **procedure** TPST($\mathbf{w}^{src}, \mathbf{s}^{src}, \mathbf{w}^{tgt}, \mathbf{s}^{tgt}$)
- 2: Align $\mathbf{w}^{src}$ to $\mathbf{w}^{tgt}$ with the Levenshtein algorithm [38], resulting in a transform $f_{align}(\cdot)$
- 3: $\mathbf{s}^{ali} \leftarrow f_{align}(\mathbf{s}^{src})$ $\triangleright \mathbf{s}^{ali}$ is a speaker sequence of length M , and may contain blank speakers $\emptyset$ due to insertion errors in the alignment
- 4: $K \leftarrow \max\{\max(\mathbf{s}^{ali}), \max(\mathbf{s}^{tgt})\}$ $\triangleright$ the maximal number of speakers in $\mathbf{s}^{ali}$ and $\mathbf{s}^{tgt}$
- 5: Initialize a cost matrix $\mathbf{C} \in \mathbb{R}^{K \times K}$
- 6: **for** $1 \leq i \leq K$ and $1 \leq j \leq K$ **do**
- 7: $C_{i,j} \leftarrow \sum_{1 \leq m \leq M} \delta(s_m^{ali} = i \text{ and } s_m^{tgt} = j)$
- 8: **end for**
- 9: Solve the assignment problem with cost matrix $\mathbf{C}$ using the Hungarian algorithm [39], resulting in a transform $f_{assign}(\cdot)$ $\triangleright$ handle speaker permutations
- 10: **for** $1 \leq m \leq M$ **do**
- 11: **if** $s_m^{ali} \neq \emptyset$ **then**
- 12: $s_m^{tra} \leftarrow f_{assign}(s_m^{ali})$ $\triangleright$ transfer the speakers from the source
- 13: **else**
- 14: $s_m^{tra} \leftarrow s_m^{tgt}$ $\triangleright$ preserve the target speaker if source speaker is unavailable
- 15: **end if**
- 16: **end for**
- 17: **end procedure**

Below we show a simple example of the inputs and output of the TPST algorithm:

Example inputs and output of the TPST algorithm

Source words:	hello good morning hi how are you pretty good
Source speakers:	1 1 1 2 2 2 2 1 1
Target words:	hello morning hi hey are you be good
Target speakers:	1 2 2 2 1 1 2 1
Transferred speakers:	1 1 2 2 2 2 1 1

3.5 LLM FINETUNING

Although the examples shown in Section 2 and Appendix B were using off-the-shelf Web APIs of commercial LLMs, finetuning the LLM specifically on the speaker diarization task is still required if we need to:

1. Reduce errors of a specific speaker diarization system;

²<https://github.com/google/speaker-id/tree/master/DiarizationLM>